

IC526 Advanced Compilers

Lec 03: Rust

- The Source Language -

Yoonseung Kim

DGIST EECS

Fall 2026

Where We Are

- Lec 01: The course overview
- Lec 02: OCaml – The development language
- Lec 03: Rust – The source language

Why Rust?

#1: TIOBE Index



About

Product

Jul 2026	Jul 2025	Change	Programming Language	
1	1			Python
2	3	▲		C
3	2	▼		C++
4	4			Java
5	5			C#
6	6			JavaScript
7	8	▲		Visual Basic
8	13	▲▲		SQL
9	15	▲▲		R
10	18	▲▲		Rust

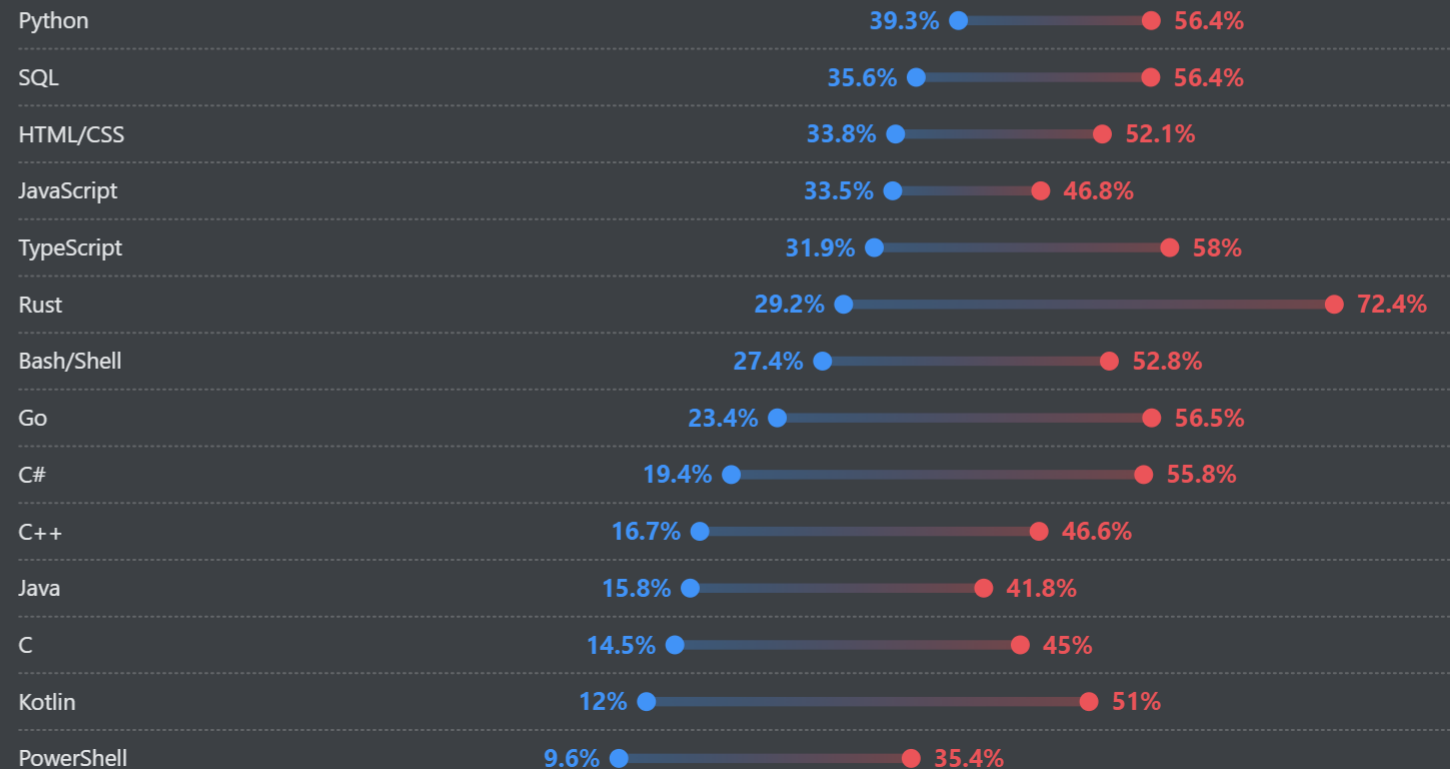
#2: StackOverflow Survey

→  survey.stackoverflow.co/2025/technology#admired-and-desired

Programming, scripting, and markup languages

Rust is yet again the most admired programming language (72%), followed by Gleam (70%), Elixir (66%) and Zig (64%). Gleam is a new addition to the list, and for good reason - developers like it!

? Which **programming, scripting, and markup languages** have you done extensive development work in over the past year, and which do you want to work in over the next year? (If you both worked with the language and want to continue to do so, please check both boxes in that row.)



How Rust Got Popular

- Rust aims to be a language for systems programming – for replacing C & C++
- The type system of Rust tells the compiler something C's type system cannot: **which pointers can refer to the same memory.**
- Memory bugs that were hard to detect are now eradicated in compile-time
 - Not after deployment; even before testing.
 - Even before compilation, the language design gives easy reasoning principle to write a safe program.

Example: Aliasing Problem

- Adding $[*b]$ twice to $[*a]$, in C++

```
void twice(int *a, const int *b) {  
    // 4 loads, 2 stores  
    *a = *a + *b;  
    *a = *a + *b;  
}
```

↓ Correct?

```
void twice(int *a, const int *b) {  
    // 2 loads, 1 store  
    *a = *a + 2 * (*b);  
}
```

```
int *t = &x;  
twice(t, t);
```

x → 3

x → 6

x → 12

$*a = *a + *b$
 $= 3 + 3$

$*a = *a + *b$
 $= 6 + 6$

x → 3

x → 9

$*a = *a + 2 * (*b)$
 $= 3 + 2 * 3$

Rust's Policy: **Exclusive** Mutable Reference

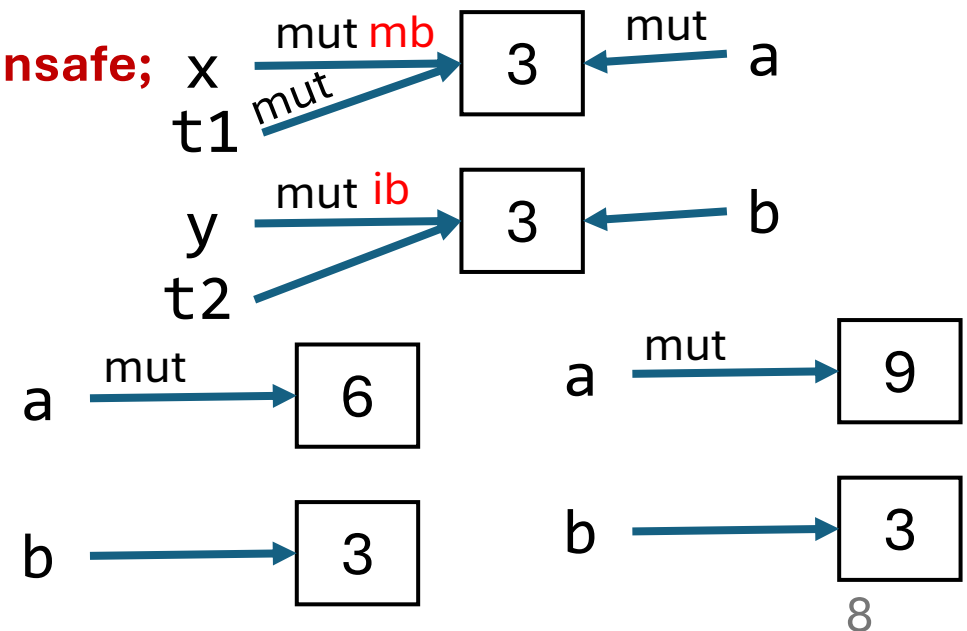
- Adding `[*b]` twice to `[*a]`, in Rust

```
fn twice(a: &mut i32, b: &i32) {  
    *a = *a + *b;  
    *a = *a + *b;  
}
```

**Rationale: usually, modifying one while others can read it is unsafe;
not a good programming pattern.**

```
void twice(int *a, const int *b) {  
    // 4 loads, 2 stores  
    *a = *a + *b;  
    *a = *a + *b;  
}
```

```
let t1: &mut i32 = &mut x;  
// [t2 = &x] or [t2 = &mut x]  
// is not allowed  
let t2: &i32 = &y;  
twice(t1, t2);
```



Our Pipeline

- Rust source
 - lexer
 - parser
 - type check & **borrow check**
 - ← **checks the policy; produces aliasing facts**
 - LLVM IR generation ← consumes them (noalias)
 - Optimizations
 - RISC-V

A Real Power of Rust: Memory Management (Our Compiler Won't Cover It, Though)

- Memory reclamation in C++: happens statically, but **unsafely**.

```
int foo() {  
    MyObj *obj = new MyObj(); // (1) Allocate space  
    bar(obj);  
    ..  
    obj->toString() // (3) Runtime error if you try to use obj  
    ..  
}  
  
int bar(MyObj *x) {  
    .. // use x  
    delete(x); // (2) The programmer thought x won't be used  
}
```

A Real Power of Rust: Memory Management (Our Compiler Won't Cover It, Though)

- Memory reclamation in Python: happens dynamically, **degrading performance**.

```
def foo():  
    MyObj obj = new MyObj(); // (1) Allocate obj new space  
    bar(obj);  
    ..  
    obj.toString(); // (3) OK since the object still exists  
    .. // At runtime, the garbage collector monitors memory usage  
  
def bar(x):  
    .. // use x  
    delete(x); // (2) Programmers don't care
```

A Real Power of Rust: Memory Management (Our Compiler Won't Cover It, Though)

- Memory reclamation in Rust: happens statically & **safely**.

```
fn foo() {  
    let mut obj : MyObj = MyObj::new(); // (1) Allocate space  
    bar(&mut obj); // (2) obj is still the sole owner of object  
    ..  
    obj.to_string(); // (4) The owner got the right back  
    ..  
    // (5) The compiler may delete object here, as no owner exists  
}  
  
fn bar(x: &mut MyObj) { // x temporarily borrows right to modify  
    .. // use x  
    delete(x); // (3) Programmers don't care,  
}
```

RustY: Our Subset of Rust

RustY: Our Subset of Rust

- We don't handle the full features of Rust.
 - Heaps
 - Concurrency
 - Complex types (string, floating point, struct, ...) (including move types)
 - Function pointers, closures, ...
- Though, we still handle:
 - Functions
 - i32, bool, array types
 - References (i.e., pointer types) in function call arguments
- Since it's a *subset*, a valid RustY program is also a valid Rust program.
 - => The standard rust compiler recognizes RustY programs.

The First Program

function declaration

Return type

```
fn add(a: i32, b: i32) -> i32 {  
    return a + b;  
}  
  
fn main() {  
    let x: i32 = add(2, 3);  
    println!("{}", x);  
}
```

variable declaration

Bindings in RustY

```
fn foo() -> i32 {  
  let x: i32 = 5;  
  let y: i32 = 7;  
  let mut z: i32 = 0; // mutable  
  z = x; // writable  
  ..  
}
```

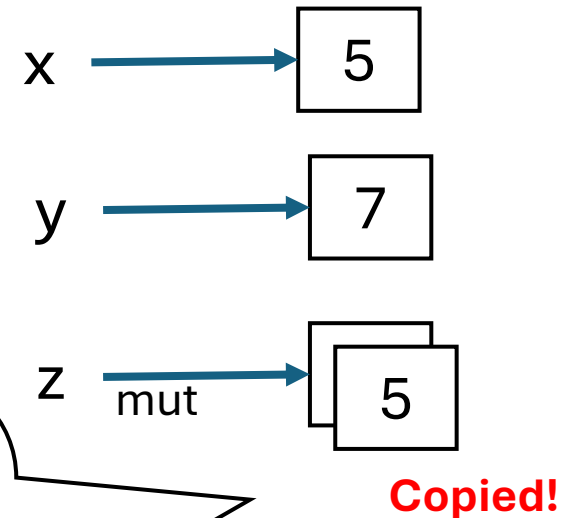
Copy and Move in **Real Rust**

b = a;

What this does depends on the type of [a].

- Copy types are duplicated — [i32], [bool], [&T].
- Move types are not: [String], [&mut T].
(After a move, using [a] is an error.)

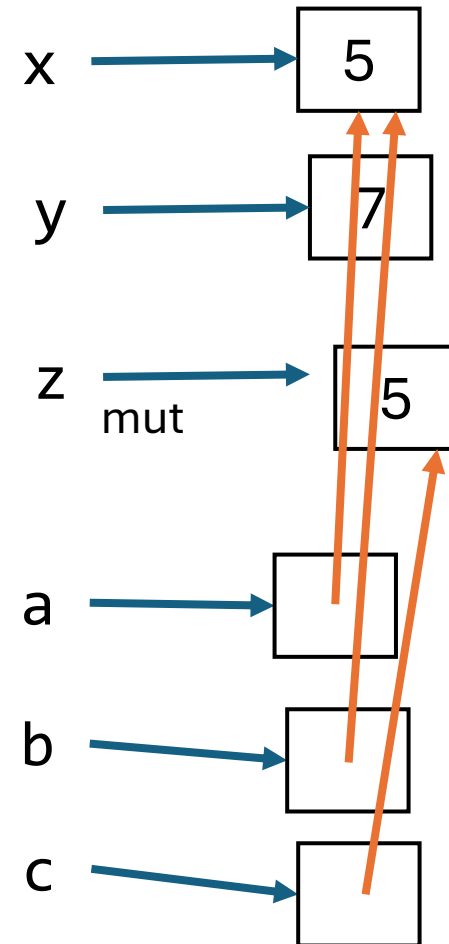
RustY has no move types. Everything is Copy.



Sharable References at Function Call

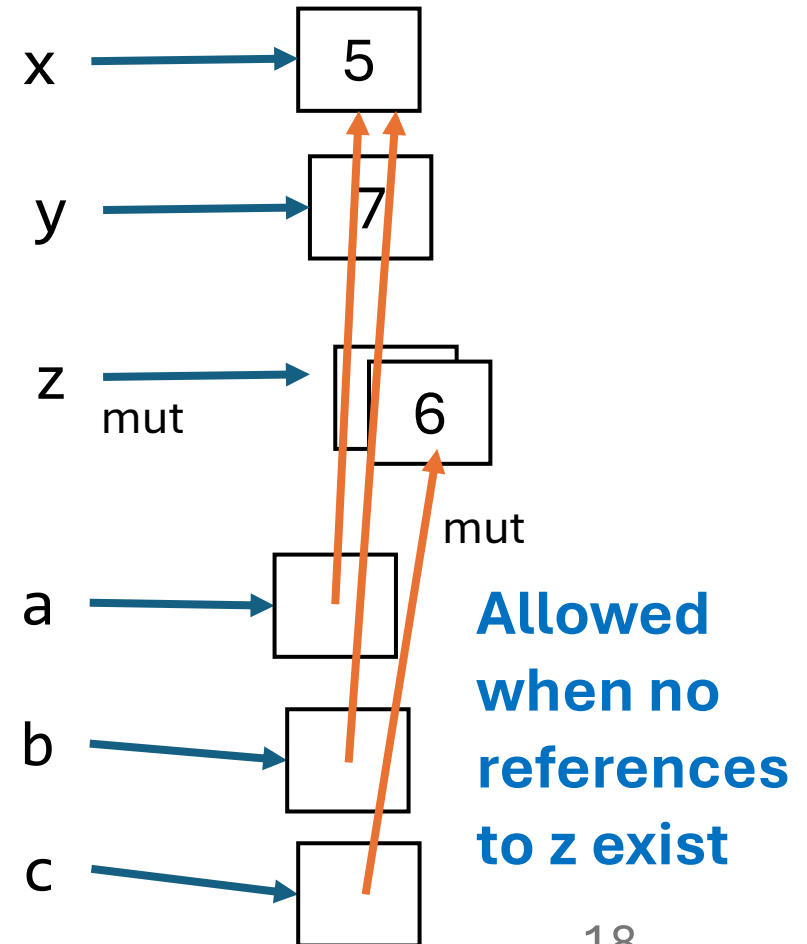
```
fn foo() -> i32 {  
  ..  
  let r : i32 = bar(&x, &x, &z);  
  println!("{}", r); // 5+5*2+5=20  
  ..  
}  
  
fn bar(a : &i32, b: &i32, c: &i32)  
  -> i32 {  
  return *a + *b + *b + *c;  
}
```

To make problems simpler, we allow
[&] only for function arguments.



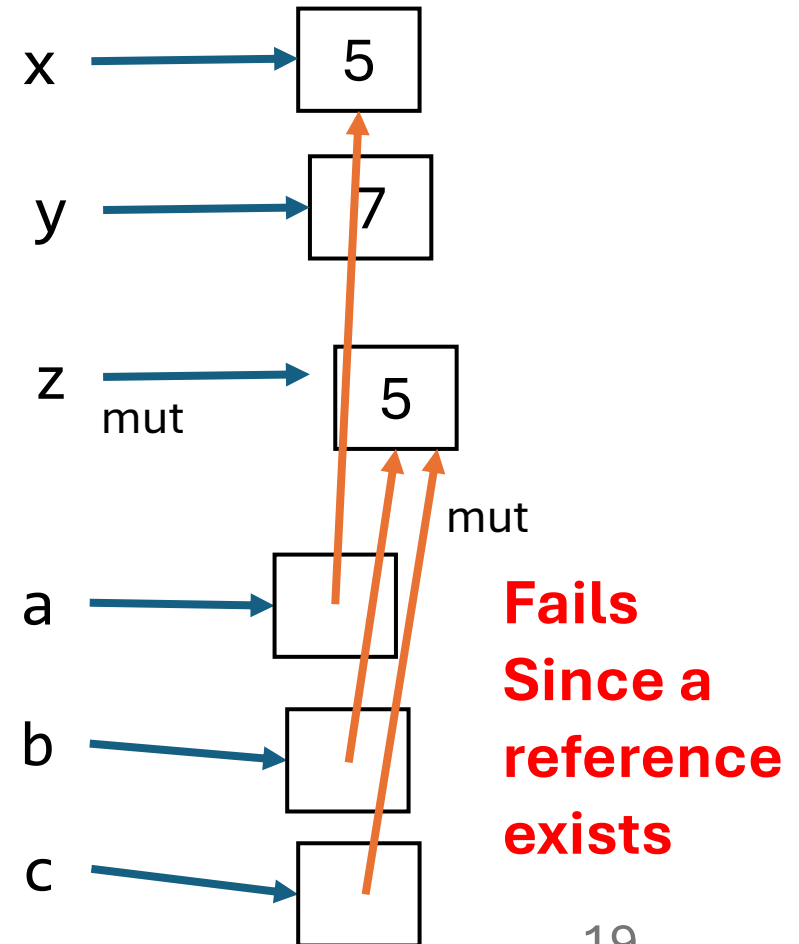
Mutable References at Function Call

```
fn foo() -> i32 {  
  ..  
  let r : i32 = bar2(&x, &x, &mut z);  
  println!("{}", r); // 5+5*2+6=21  
  ..  
}  
  
fn bar2(a : &i32, b: &i32, c: &mut i32)  
  -> i32 {  
  *c = *c + 1;  
  return *a + *b + *b + *c;  
}
```



Mutable References at Function Call

```
fn foo() -> i32 {  
  ..  
  let r : i32 = bar2(&x, &z, &mut z);  
  println!("{}", r);  
  ..  
}  
  
fn bar2(a : &i32, b: &i32, c: &mut i32)  
  -> i32 {  
  *c = *c + 1;  
  return *a + *b + *b + *c;  
}
```

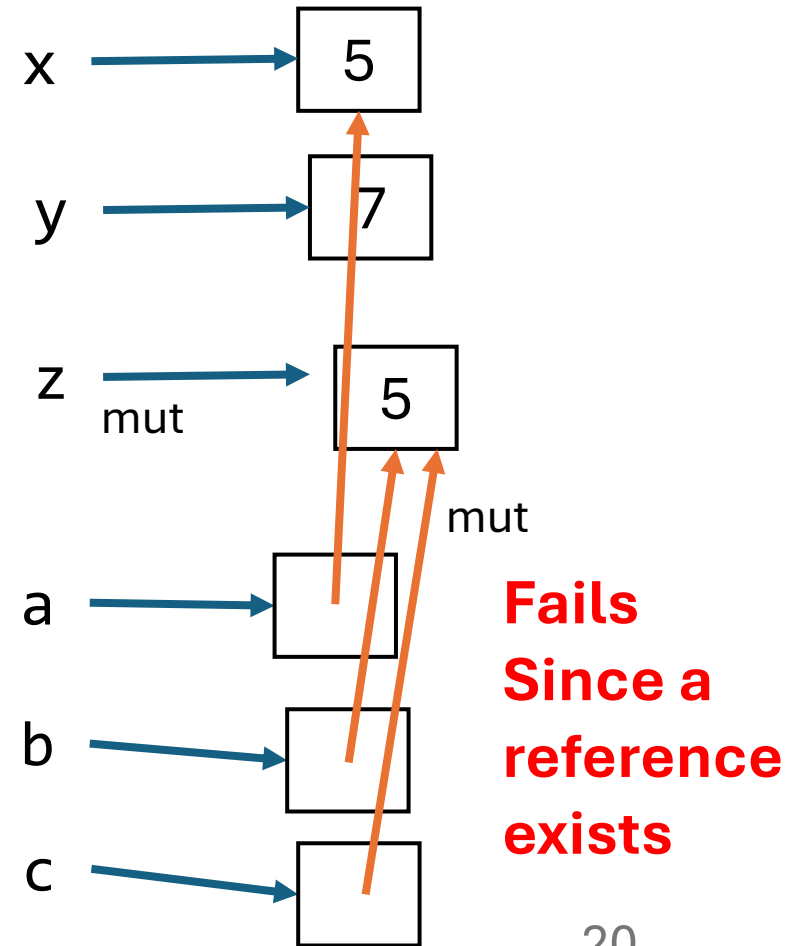


Mutable References at Function Call

```
fn foo() -> i32 {  
  ..  
  let r : i32 = bar2(&x, &z, &mut z);  
  println!("{}", r);  
  ..  
}  
  
fn bar2(a : &i32,  
  -> i32 {  
  *c = *c + 1;  
  let t : i32 = *b;  
  return *a + t + t + *c;  
}
```

Philosophy:

It's clearer to reason about programs
when you **disallow modifying a value**
while others see it.



Types

N ::= 0 | 1 | 2 | 3 | ...
| MAX_USIZE (*constant*)

T ::= i32
| usize (*arr idx*)
| bool
| () (*unit*)
| [T ; N] (*arr*)

ArgT ::= T | &T | &mut T
(* funarg types *)

```
fn arr_sum(x : &mut [i32; 3], y: &[i32; 3])  
/* you can omit [-> ()] */ {  
    let mut i : usize = 0;  
    while i < 3 {  
        (*x)[i] = (*x)[i] + (*y)[i];  
        i = i + 1;  
    }  
}  
  
fn main() {  
    let mut arr1 : [i32; 3] = [1, 2, 3];  
    let mut arr2 : [i32; 3] = [6, 5, 4];  
    arr_sum(&mut arr1, &arr2);  
    println!("{}", arr1[0]);  
    ...  
}
```

Expressions

$E ::= n$ (* number literals *)
| x (* variable *)
| `true` | `false`
| $[E, \dots, E]$ (* array *)
| $*x$ (* deref *)
| $\text{uop } E$ (* unary op (-, !) *)
| $E \text{ bop } E$ (* binary op (+, <, ...) *)
| $E[E]$ (* array access *)
| (E) (* parenthesis *)
| $f(\text{ArgE}, \dots, \text{ArgE})$

$\text{ArgE} ::= E \mid \&E \mid \&\text{mut } E$

```
fn arr_sum(x : &mut [i32; 3], y: &[i32; 3])  
/* you can omit [-> ()] */ {  
    let mut i : usize = 0;  
    while i < 3 {  
        (*x)[i] = (*x)[i] + (*y)[i];  
        i = i + 1;  
    }  
}  
  
fn main() {  
    let mut arr1 : [i32; 3] = [1, 2, 3];  
    let mut arr2 : [i32; 3] = [6, 5, 4];  
    arr_sum(&mut arr1, &arr2);  
    println!("{}", arr1[0]);  
    ...  
}
```

Statements

```
S ::= let x:T = E ;  
      | let mut x:T = E ;  
      | E ; (* function call *)  
      | E = E ; (* x = 3 / *x = *y + 1 *)  
      | if E B  
      | if E B else B  
      | while E B  
      | println!("{}", E)  
      | return E;
```

with

```
B ::= { S* }
```

```
fn arr_sum(x : &mut [i32; 3], y: &[i32; 3])  
/* you can omit [-> ()] */ {  
    let mut i : usize = 0;  
    while i < 3 {  
        (*x)[i] = (*x)[i] + (*y)[i];  
        i = i + 1;  
    }  
}  
  
fn main() {  
    let mut arr1 : [i32; 3] = [1, 2, 3];  
    let mut arr2 : [i32; 3] = [6, 5, 4];  
    arr_sum(&mut arr1, &arr2);  
    println!("{}", arr1[0]);  
    ...  
}
```

Function & Prog

$F ::= \text{fn } f(x : \text{ArgT}, \dots) \rightarrow T \{ S^* E? \}$
 $\quad | \text{fn } f(x : \text{ArgT}, \dots) \{ S^* \}$

$P ::= F^*$

```
fn arr_sum(x : &mut [i32; 3], y: &[i32; 3])
/* you can omit [-> ()] */ {
    let mut i : usize = 0;
    while i < 3 {
        (*x)[i] = (*x)[i] + (*y)[i];
        i = i + 1;
    }
}

fn main() {
    let mut arr1 : [i32; 3] = [1, 2, 3];
    let mut arr2 : [i32; 3] = [6, 5, 4];
    arr_sum(&mut arr1, &arr2);
    println!("{}", arr1[0]);
    ...
}
```


Practice

- `find_max.rs`
- `bubble_sort.rs`
- `fib.rs`